



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

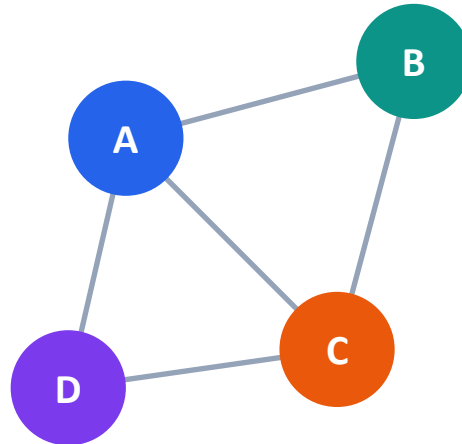


CSA0308 – DATA STRUCTURES

Application of suitable data structures and ADTs for
efficient problem solving

Course Outcome: CO1-AT2-Technical Presentation

GUIDED BY:
Dr. RAJARAM



PRESENTED BY:
ABISHANA S K
(192511242)



ENGINEER TO EXCEL



SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)



SIMATS ENGINEERING
ASSESSMENT TOOL 2 - Technical Presentation

COURSE NAME: Data Structures

COURSE CODE: CSA0308

Register Number	192511242
Name	Abishana S K

Assessment Tool Weightage: 30%

COURSE OUTCOME COVERED

CO1: Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)

Questions: 4 **EACH QUESTIONS CARRIES:** 25 MARKS **Total Marks:** 100 **Duration:** ONE HOUR

S. No.	List of Questions
1	A student management system needs to store the marks of 1000 students efficiently. Apply the concepts of primitive and non-primitive data structures to select an appropriate structure for storing and managing the data and prepare a technical presentation justifying your choice based on the efficiency of data structure.
2	Choose a suitable linear or non-linear data structure for a navigation system that stores city connections and finds shortest routes between locations. Prepare a technical presentation explaining why it is appropriate.
3	A web browser supports navigating backward through previously visited pages. Recommend a suitable ADT and prepare a technical presentation explaining how it improves performance efficiency.
4	A digital library allows searching books using ISBN numbers quickly among 50,000 records. Select the most suitable data structure and <u>Prepare</u> a technical presentation justifying using time complexity comparison.

Code of Conduct:

I Abishana S K (192511242) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Abishana S K



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well- analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Q1: STUDENT MANAGEMENT SYSTEM

Q1



Store marks of 1000 students → Array of student records

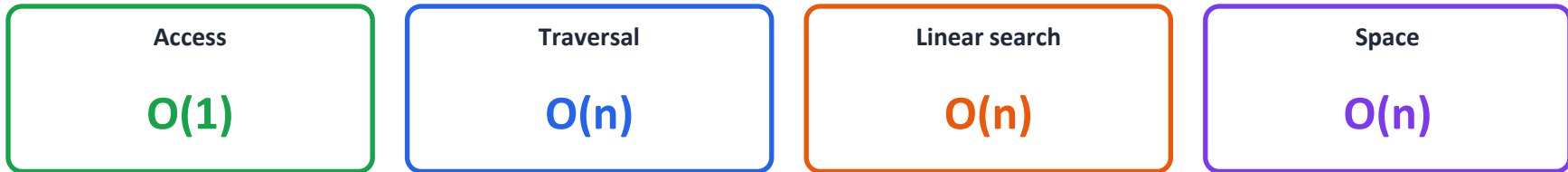
index 0	index 1	index 2	index 3	index ...	index 999
101 Asha 85	102 Ravi 78	103 Meena 92	104 Karan 67	...	1100 Divya 88

Address of $S[i] = \text{base} + i \times \text{size} \rightarrow \text{direct (index-based) access}$

- The system must store and manage marks for 1000 students.
- We choose an array of student records containing ID, name and marks.
- Each record is stored at an index, allowing direct access and updates.
- The fixed number of students makes an array a simple and efficient choice.

Q1: SOLUTION & COMPLEXITY

Q1

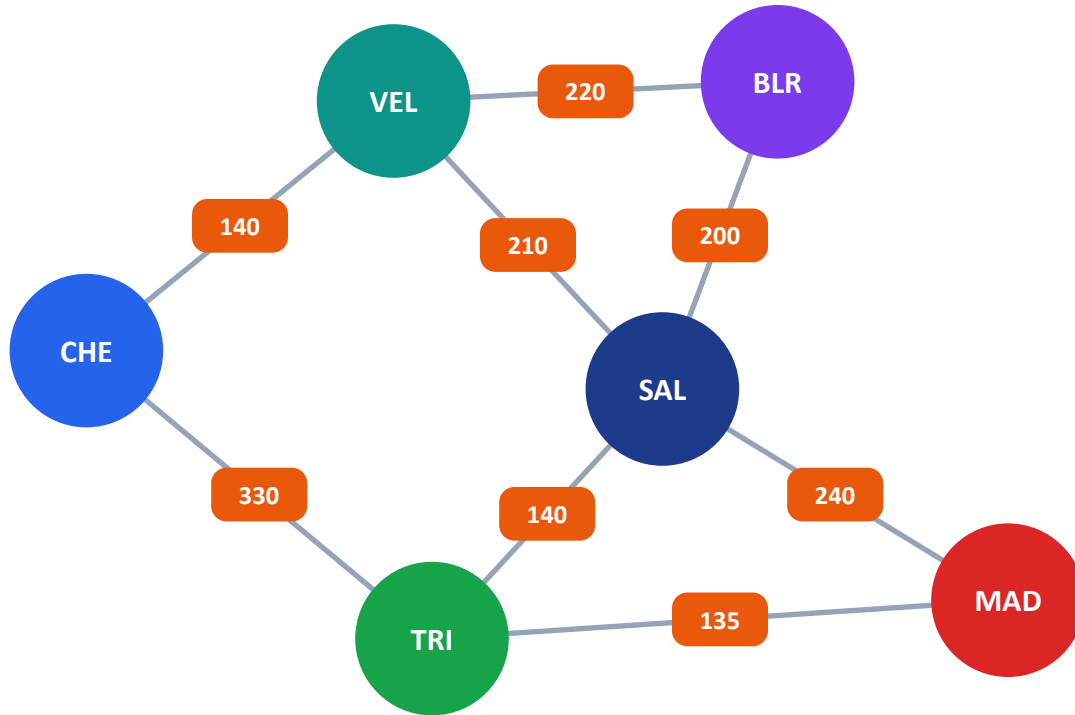


Justification

- Marks are fetched or updated instantly by roll/index – $O(1)$
- Fixed 1000 records suit a static array; memory is contiguous with no extra pointers
- Traversal for reports / averages is a single $O(n)$ pass

Q2: NAVIGATION SYSTEM

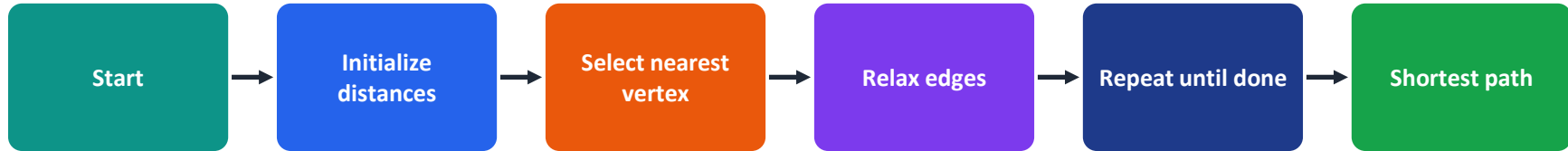
Q2



- The system must represent cities and their interconnected roads.
- We choose a weighted graph, where vertices represent cities and edges represent roads.
- Edge weights represent distance, travel time, or cost.
- This structure allows shortest-path algorithms to find efficient routes.

Q2: SHORTEST ROUTE SOLUTION

Q2



➤ Dijkstra's algorithm finds the shortest route when edge weights are non-negative.

➤ It repeatedly selects the nearest vertex and relaxes its neighbouring edges.

➤ An adjacency list with a binary heap provides efficient graph processing.

➤ Time: $O((V+E) \log V)$ and space: $O(V+E)$.

Time

$O((V+E) \log V)$

Space

$O(V+E)$

Binary-heap priority queue; adjacency list

Justification

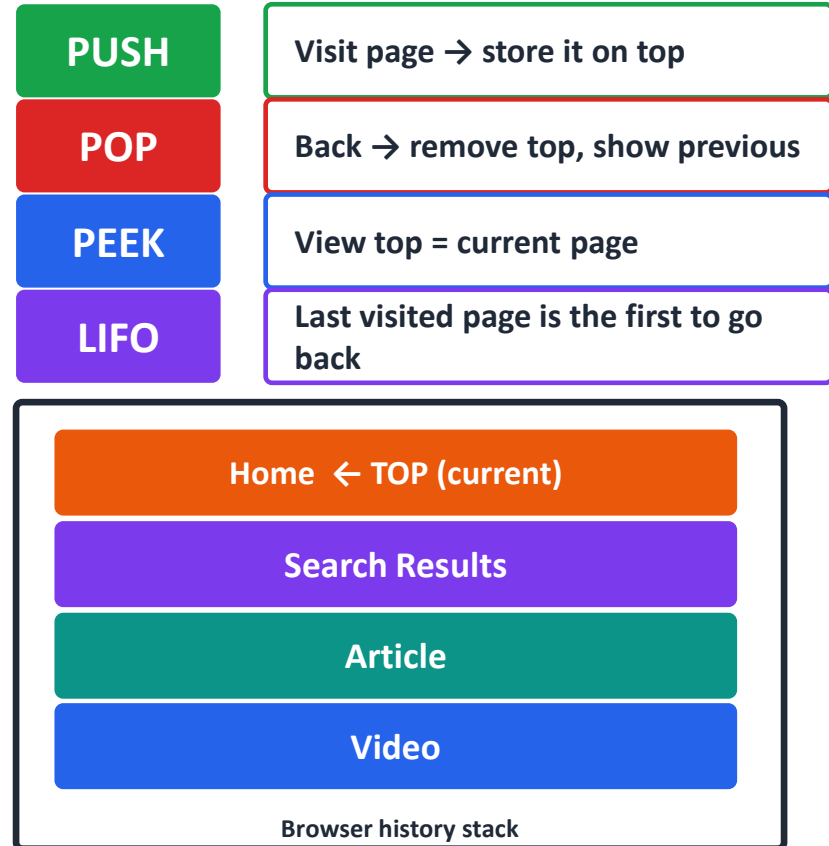
- Road distances are non-negative → Dijkstra's greedy choice is correct and optimal
- Efficient for sparse road networks with an adjacency list + min-heap

Q3: BROWSER BACKWARD NAVIGATION

Q3



- The browser needs to return to the most recently visited page first.
- We choose the Stack ADT, which follows the LIFO principle.
- Visited pages are added using Push and removed using Pop when Back is pressed.
- This makes Stack a natural fit for backward navigation.





ENGINEER TO EXCEL

Q3: SOLUTION & EFFICIENCY

Q3



Push

$O(1)$

Pop

$O(1)$

Peek

$O(1)$

Space

$O(n)$

Stack (LIFO) ✓

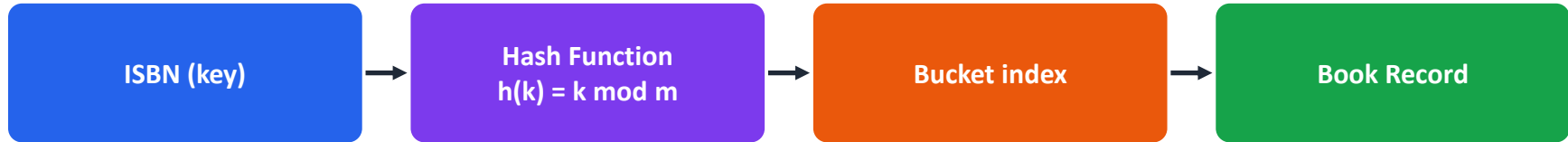
Last visited page is on top, so Pop returns exactly the previous page in $O(1)$

Queue (FIFO) ✗

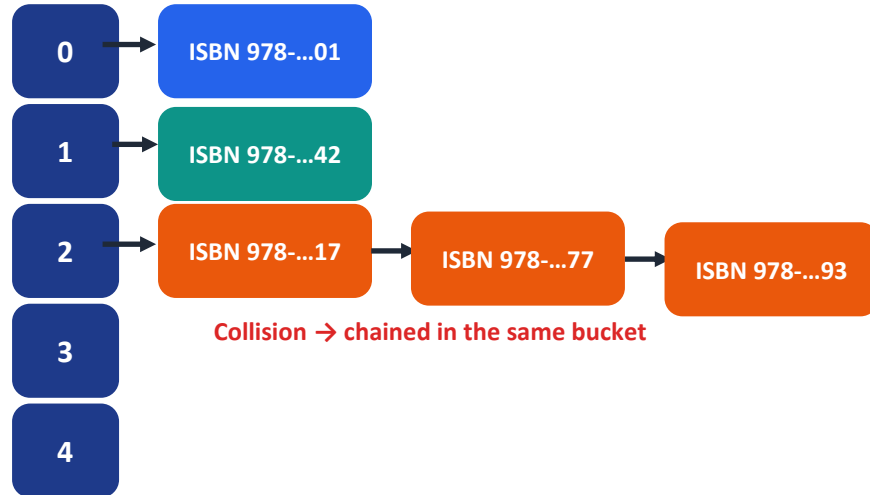
Deque returns the oldest page (e.g. Home), not the page just visited – wrong behaviour

Q4: DIGITAL LIBRARY ISBN SEARCH

Q4



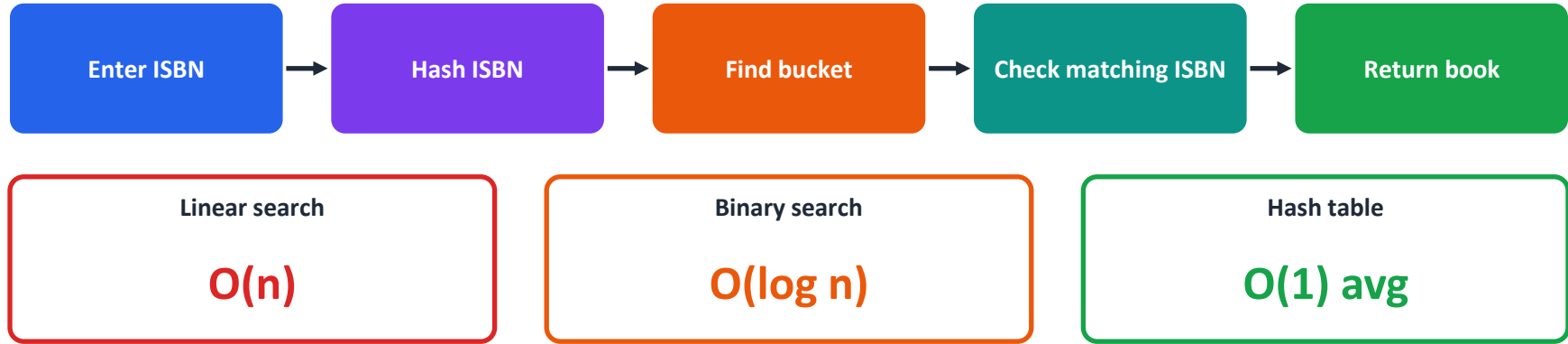
Hash table with chaining



- The system must find a book quickly among 50,000 records.
- We choose a Hash Table, using ISBN as the unique search key.
- A hash function maps the ISBN to a bucket for fast lookup.
- Collisions are handled using chaining, giving $O(1)$ average-case search.

Q4: SOLUTION & COMPLEXITY

Q4



$O(1)$ is AVERAGE-case; worst case becomes $O(n)$ if many ISBNs collide in one bucket

Why Hash Table?

Frequent exact-match ISBN lookups need the fastest search; a good hash function keeps chains short and no sorting is required

COMPARATIVE ANALYSIS

Problem	Data Structure	Main Operation	Complexity
Student Management	Array / Student Records	Access	$O(1)$
Navigation	Graph + Dijkstra	Shortest Path	$O((V+E) \log V)$
Browser History	Stack	Push / Pop	$O(1)$
ISBN Search	Hash Table	Search	$O(1)$ average

Each structure is chosen for the operation the problem uses most.

THANK YOU